**Department of Computer Science and Engineering**
(AI and ML)
B. Tech CSE - 5th Semester

**UE24CS351A**
**DATABASE MANAGEMENT SYSTEMS**

**PROJECT REPORT ON**

# IQAC Audit and Compliance Management System

*A database-driven evidence, review and scoring portal for institutional quality audits*

**Submitted by**

| Name | SRN |
|---|---|
| Dabbugunta Venya Anand | PES1UG24AM074 |
| Devika N | PES1UG24AM080 |

Class of Prof. Smitha GR
Lab faculty : Smitha GR

## Abstract

The IQAC Audit and Compliance Management System centralizes evidence collection, staged verification and score reporting for an institution's Internal Quality Assurance Cell. Faculty members create structured evidence records, Heads of Department verify faculty submissions in their own department, and an IQAC Auditor performs final review. MySQL stores the transactional core, including users, departments, audit cycles, criteria, submissions, department metrics, audit logs and score aggregates. MongoDB stores flexible evidence documents whose fields vary by activity type, such as research papers, patents, workshops, placement data, budgets and laboratory upgrades. The system uses database constraints, triggers, stored procedures and role-based access control to prevent invalid workflow transitions, preserve an audit trail and calculate department scores. A Next.js portal integrates both databases and offers reports and role-aware statistics. The implementation demonstrates an end-to-end DBMS project with relational and NoSQL design, DDL, DML, advanced SQL, backend integration and authentication.


## Contents


| No. | Section |
|---|---|
| 1 | Introduction, problem statement and short description |
| 2 | User requirements specification and technology justification |
| 3 | E-R model, entity list, attribute types and E-R diagram |
| 4 | Relational model, relation list and schema diagram |
| 5 | SQL DDL statements |
| 6 | SQL DML statements |
| 7 | Advanced SQL concepts: triggers, procedure and queries |
| 8 | NoSQL database commands and MongoDB validation |
| 9 | Next.js project tree structure |
| 10 | Results, resulting tables and testing |
| 11 | Sample application code and role-based demonstration |
| 12 | Evaluation rubric completion |
| 13 | Conclusion, future scope and references |

# 1 Introduction

Institutions collect research, training, placement, finance and infrastructure evidence throughout an academic year for IQAC, NAAC, NBA and NIRF-related review. When departments use separate spreadsheets, emails and paper records, evidence is difficult to trace, duplicate claims are hard to detect and the final report becomes a year-end manual exercise. This project provides one controlled record system for that workflow.

## 1.1 Problem Statement

The system must record evidence from multiple departments, retain enough structured information for scoring and reporting, allow evidence details to vary by activity type, and ensure that only the correct reviewer can change a record at each stage. The solution must also preserve review history and provide a concise view of pending work.

## 1.2 Short Description

The project is a web-based audit workspace in which Faculty members enter evidence, HODs perform department-level verification, and an IQAC Auditor completes final review. It combines a MySQL workflow database with MongoDB evidence documents and exposes the process through a role-based Next.js portal.

## 1.3 Objectives

- Maintain department, user, audit-cycle and criteria-master data in a normalized relational database.
- Store flexible evidence documents while preserving a one-to-one link with the corresponding MySQL record.
- Enforce the workflow Draft -> Submitted -> Verified -> Approved, with controlled rejection and reopening paths.
- Provide role-based access for Faculty, HOD, Auditor and Admin.
- Compute an explainable weighted department score out of 100 using approved leaf metrics only.
- Provide reports and a review queue that identifies the next responsible role.

# 2 User Requirements and Technology Justification

| Role | Main requirement | System support |
|---|---|---|
| Faculty | Create evidence and follow its review status | Own Draft/Rejected faculty records; submit for HOD review |
| HOD | Review faculty work within a department and record department metrics | Department-scoped visibility; verify/reject faculty work; create metrics |
| Auditor | Perform final review across departments | Approve/reject verified faculty records and review department metrics |
| Admin | Institutional oversight | View data and manage cycle lifecycle without impersonating an Auditor |

### 2.1 Technology Stack

| Technology | Use | Justification |
|---|---|---|
| MySQL 8 | Transactional core | Foreign keys, transactions, CHECK constraints, triggers, functions and stored procedures fit the controlled workflow. |
| MongoDB | Evidence documents | Activity-specific nested fields and arrays such as authors, attachments, equipment and budget heads require schema flexibility with validation. |
| Next.js | Web portal and API | Provides a single JavaScript application for the browser interface and authenticated server routes. |
| Node.js MongoDB driver and mysql2 | Database integration | Server-side connections keep credentials away from browser users. |
| Git and GitHub | Source control | Tracks the SQL scripts, application changes and documentation. |

MongoDB is not used as a replacement for MySQL. MySQL remains the source of truth for identities, ownership, workflow state, scores and audit logs. MongoDB stores the detailed evidence payload linked by a fixed ObjectId. This combines transaction safety with flexible evidence structure.

## 3 E-R Model

The conceptual model contains eight core entities. Departments have users; users participate in audit cycles through faculty submissions or department metrics; criteria form a self-referencing hierarchy; and each workflow change produces a verification log. Score aggregates are derived per department and audit cycle. The detailed Chen ER diagram is supplied separately as IQAC_ER_CHEN.pdf.

Entity classification: Departments, User_Auth_Roles, Audit_Cycles, Criteria_Metrics_Master, Faculty_Submissions, Department_Metrics, Verification_Logs and Score_Aggregates are all strong entities in this project. There are no weak entities because every relation has its own primary key.

| Entity | Primary key | Important attributes | Relationships |
|---|---|---|---|
| Departments | dept_id | dept_name, dept_code | Has users; owns metrics and scores |
| User_Auth_Roles | user_id | name, email, role, dept_id | Owns faculty submissions; acts in logs |

| Entity | Primary key | Important attributes | Relationships |
|---|---|---|---|
| Audit_Cycles | cycle_id | name, start_date, end_date, status | Contains records and aggregates |
| Criteria_Metrics_Master | metric_id | criterion, max_score, weightage, target_role | Self-parent hierarchy; classifies records |
| Faculty_Submissions | submission_id | title, DOI/ISBN, status, score | Belongs to faculty, cycle and metric |
| Department_Metrics | dept_metric_id | metric_value, status | Belongs to HOD department, cycle and metric |
| Verification_Logs | log_id | old status, new status, remarks, time | Records a status event |
| Score_Aggregates | aggregate_id | total_weighted_score, generated_at | Snapshot for department and cycle |

## 3.2 ER Diagram Snapshot

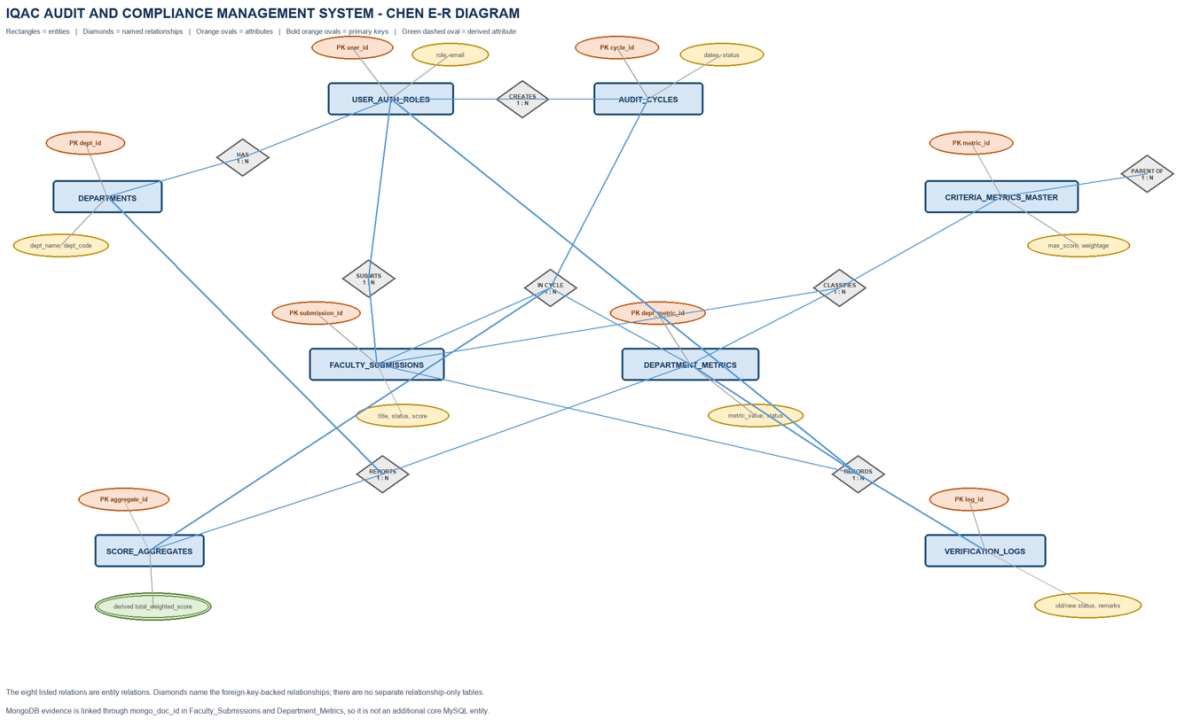



*Figure 1. E-R diagram for the eight core entities.*

## 3.1 Non-Simple Attribute

Non-simple attributes: full_name is represented as first_name plus last_name; evidence authors and attachments are multivalued arrays in MongoDB; criteria parent_metric_id represents the recursive hierarchy. The weighted departmental score is derived from approved leaf metrics and retained as a snapshot.

# 4 Relational Model

The relational implementation uses the following eight core relations. These relations represent the conceptual entities in the E-R model and store the normalized data needed for the audit workflow, verification history and score calculation.

| Relation | Key and references | Purpose |
|---|---|---|
| Departments | PK dept_id | Department master |
| User_Auth_Roles | PK user_id; FK dept_id | User identity and role |
| Audit_Cycles | PK cycle_id | Audit period and lifecycle |
| Criteria_Metrics_Master | PK metric_id; FK parent_metric_id | Criteria tree and scoring policy |
| Faculty_Submissions | PK submission_id; FK faculty_id, cycle_id, metric_id | Faculty activity record |
| Department_Metrics | PK dept_metric_id; FK hod_id, dept_id, cycle_id, metric_id | Department-level record |
| Verification_Logs | PK log_id; FK actor and one record ID | Append-only status history |
| Score_Aggregates | PK aggregate_id; FK dept_id, cycle_id | Calculated department score |

## 4.1 Key Constraints

- Primary keys identify all entity rows and foreign keys use restrictive delete behavior for audit safety.
- Unique constraints protect department names/codes and prevent duplicate references where applicable.
- CHECK constraints validate dates, score ranges, email/URL patterns and ObjectId formats.
- A trigger prevents a criteria row from directly using itself as parent; this rule is enforced in triggers rather than an invalid CHECK involving AUTO_INCREMENT insertion behavior.
- Verification_Logs requires exactly one of submission_id or dept_metric_id for each history event.

## 4.2 Schema Diagram

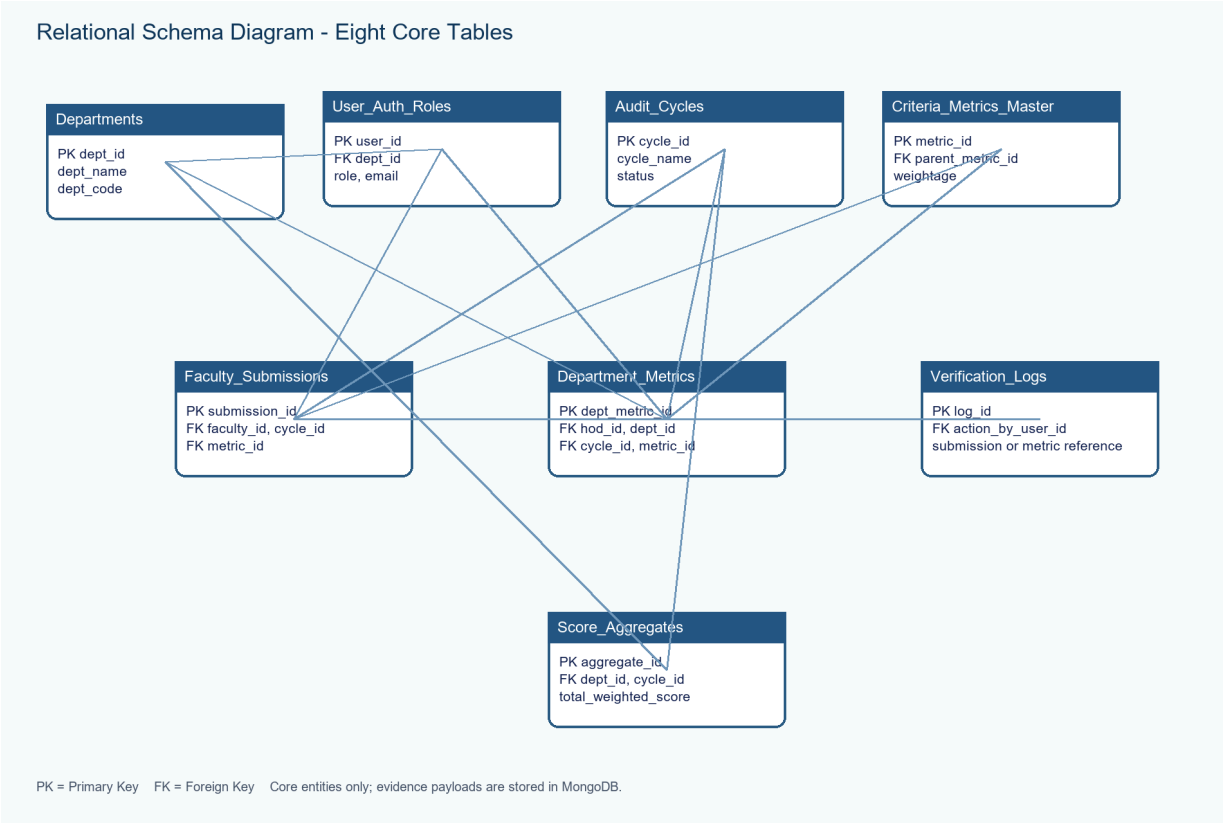



*Figure 2. Logical relational schema diagram with primary and foreign-key relationships.*

# 5 SQL DDL Statements

01_schema.sql creates the MySQL database and the eight core tables. It defines primary keys, foreign keys, unique constraints and validation checks. 04_seed_data.sql then creates three departments, seven users, two audit cycles, twenty criteria rows, six faculty submissions, three department metrics and their initial logs.

```
-- ========================================================================
-- IQAC Audit & Compliance Management System
-- STEP 2: MySQL DDL  (MySQL 8.0.16+ required, CHECK constraints enforced)
-- Run:  mysql -u root -p < 01_schema.sql
-- ========================================================================

DROP DATABASE IF EXISTS iqac_db;
CREATE DATABASE iqac_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE iqac_db;

-- ------------------------------------------------------------------------
-- 1. Departments
-- ------------------------------------------------------------------------
CREATE TABLE Departments (
    dept_id    INT          NOT NULL AUTO_INCREMENT,
    dept_name  VARCHAR(100) NOT NULL,
    dept_code  VARCHAR(10)  NOT NULL,
    CONSTRAINT pk_departments      PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_name        UNIQUE (dept_name),
    CONSTRAINT uq_dept_code        UNIQUE (dept_code)
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- 2. User_Auth_Roles
--    full_name (composite attribute) is split into first_name + last_name
-- ------------------------------------------------------------------------
```

*Figure 3. SQL DDL source capture from 01_schema.sql. Refer 01_schema.sql for full code.*

# 6 SQL DML Statements

05_dml_queries.sql demonstrates data manipulation and retrieval operations. It contains insertion, update, deletion, joins, nested/correlated queries, a recursive criteria query and reporting queries. The queries use the same seeded records demonstrated in the application.

```
-- ------------------------------------------------------------------------
-- A1. INSERT: a faculty member creates a Draft submission, and the audit
--     trail records it
-- ------------------------------------------------------------------------
SELECT 'A1  INSERT a draft submission + its log entry' AS demo;
START TRANSACTION;

INSERT INTO Faculty_Submissions
    (faculty_id, metric_id, cycle_id, title, doi_or_isbn, evidence_url, status)
VALUES
    (5, 6, 1, 'Practical Guide to Advanced Database Systems',
     '978-81-2345-678-9', 'https://drive.example.edu/book_chapter.pdf', 'Draft');

SET @new_sub = LAST_INSERT_ID();

INSERT INTO Verification_Logs
    (submission_id, dept_metric_id, action_by_user_id, previous_status, new_status, remarks)
VALUES
    (@new_sub, NULL, 5, NULL, 'Draft', 'Draft created');

SELECT submission_id, faculty_id, title, status FROM Faculty_Submissions WHERE submission_id = @new_sub;
SELECT log_id, submission_id, new_status, remarks FROM Verification_Logs WHERE submission_id = @new_sub;

-- ------------------------------------------------------------------------
-- A2. INSERT: an HOD enters a department metric (placement percentage)
-- ------------------------------------------------------------------------
SELECT 'A2  INSERT a department metric' AS demo;
INSERT INTO Department_Metrics
```

*Figure 4. SQL DML and query source capture from 05_dml_queries.sql. Refer 05_dml_queries.sql for full code*

# 7 Advanced SQL Concepts

## 7.1 Trigger Controlled Workflow

| Event | Database rule |
|---|---|
| Faculty insert/update | Owner, active cycle, eligible leaf criterion, duplicate evidence and valid transition are checked. |
| HOD verification | Only the HOD for the record's department may move a faculty record from Submitted to Verified. |
| Auditor approval | Only the Auditor may approve a Verified faculty record and assign a score within its maximum. |
| Approved record update | Approved records are protected from later tampering. |
| Status change | A verification-log row records actor, old status, new status, time and remarks. |

## 7.2 Stored Procedure and Score Policy

sp_recalculate_scores(cycle_id) computes a weighted score out of 100. Only Approved leaf metrics contribute. Faculty scores are summed per metric and capped at the metric maximum. Department metric values are percentages converted to their metric maximum. Each leaf contribution is multiplied by its configured weightage; missing work contributes zero and weight is not redistributed. The original seed produces CSE 31.25, ECE 19.50 and ME 0.00.

## 7.3 Query Coverage

| Query category | Example in project |
|---|---|
| Join query | Department-wise approved faculty submissions with faculty name and criterion. |
| Nested/correlated query | Departments whose approved work is above or below aggregate comparisons. |
| Recursive CTE | Criteria hierarchy traversal from parent criterion to leaf metric. |
| Report query | Pending work grouped by department and current review stage. |
| Aggregation | Weighted score and approval percentage by audit cycle. |

# 8 NoSQL Database Commands

MongoDB contains faculty_evidence and department_evidence. Each document has a stable ObjectId matching mongo_doc_id in MySQL, a reverse MySQL identifier and a validated type-specific details object. Faculty evidence can represent a research paper, patent, workshop, FDP, book chapter or mentorship. Department evidence can represent a placement, budget, lab upgrade or infrastructure record.

| Feature | Implementation |
|---|---|
| Schema validation | $jsonSchema validation and oneOf branches validate each evidence type. |
| One-to-one link | Unique indexes on mysql_submission_id and mysql_dept_metric_id prevent duplicate document links. |
| Search and filters | Indexes support owner/cycle lookup, author lookup and text search over title, abstract and keywords. |
| CRUD | The portal creates and updates the linked MySQL and MongoDB evidence records through its backend. |
| Failure handling | If MongoDB is temporarily unavailable, the committed MySQL record remains traceable and synchronization can be retried safely. |

```
// ======================================================================
// IQAC Audit & Compliance Management System
// STEP 3: MongoDB creation  (run with mongosh)
//
//   mongosh "mongodb://localhost:27017" --file 02_mongo_setup.js
//
// Database : iqac_nosql
// Purpose  : stores the FLEXIBLE evidence details of each submission.
//            MySQL keeps the structured/scoring data; each MySQL row points
//            here through mongo_doc_id, and each Mongo document points back
//            through mysql_submission_id / mysql_dept_metric_id.
// ======================================================================

const dbName = "iqac_nosql";
const iq = db.getSiblingDB(dbName);

// Start clean so the script can be re-run safely
iq.faculty_evidence.drop();
iq.department_evidence.drop();

// ----------------------------------------------------------------------
// Shared sub-schema: list of attached evidence files
// ----------------------------------------------------------------------
const attachmentsSchema = {
  bsonType: "array",
  items: {
    bsonType: "object",
    required: ["file_name", "url"],
    properties: {
      file_name:   { bsonType: "string" },
      url:         { bsonType: "string", pattern: "^http" },
      uploaded_at: { bsonType: "date" }
    }
  }
};
```

*Figure 5. MongoDB collection, validation and index setup from 02_mongo_setup.js.*

# 9 Next.js Project Tree Structure

Next.js Project Tree

```
app/
  api/[...path]/route.js
  page.js
  statistics.js
lib/
  policy.js
  service.js
  statistics.js
scripts/
  setup.mjs
  bootstrap.mjs
  mongo-tests.mjs
01_schema.sql
02_mongo_setup.js
04_seed_data.sql
06_triggers.sql
09_score_calculation.sql
10_reports.sql
11_statistics_queries.sql
README.md
SETUP.md
LIFECYCLE.html
```

*Figure 6. Project tree showing the application, SQL scripts, tests and documentation.*

### 9.1 Web Application and RBAC

The Next.js application exposes authenticated API routes for sign-in, dashboard data, records, status transitions, evidence, logs, reports, score refresh and statistics. A signed server-side session identifies the actor; backend checks and database triggers independently enforce access. The browser never receives the database setup account.

| Role | Can read | Can change |
|---|---|---|
| Faculty | Own faculty submissions | Own Draft/Rejected submission; submit for review |
| HOD | Faculty and department records in own department | Verify/reject Submitted faculty records; create own department metrics |
| Auditor | Records across departments | Review department metrics; approve/reject Verified faculty records |
| Admin | Institution-wide oversight | Manage audit cycle lifecycle; no Auditor approval action |

### 9.2 End-to-End Lifecycle

A faculty member creates a Draft with a MySQL record and an associated MongoDB evidence document. On submission, the record enters the HOD queue. The relevant HOD verifies or rejects it. A Verified faculty record enters the Auditor queue; the Auditor approves it with a score or returns it for correction. Every transition adds a review log. The separate LIFECYCLE.html walkthrough maps this sequence to the current source code.

## 10 Results and Resulting Tables

The Reports page provides approved faculty-submission reports, pending-review reports and department-score reports by audit cycle, with CSV export. Access is scoped on the server: a Faculty member sees own submissions, an HOD sees only the department and an Auditor or Admin can compare departments.

The Statistics landing page presents current status distribution, approval percentage, department backlog, monthly creation activity and department scores. The practical review-triage rule is that Submitted faculty work waits for the HOD, while Verified faculty work and Submitted/Verified department metrics wait for the Auditor. This makes the dashboard actionable rather than showing only a single generic pending count.

| Seed demonstration queue | Record |
|---|---|
| HOD CSE review | Machine Learning Applications in Medical Diagnostics - Submitted |
| Auditor faculty review | Low-Cost IoT Device for Soil Moisture Monitoring - Verified |
| Auditor department reviews | ECE budget metric - Submitted; CSE laboratory-upgrade metric - Verified |

.

## 10.1 Testing and Resulting Data

The project includes unit tests for password handling, policy rules, evidence validation, reports, score calculations, SQL-script parsing and statistics filters. Live SQL quality checks exercise schema import, trigger behavior, direct self-parent rejection, workflow enforcement, score calculation and restricted application permissions. MongoDB tests verify schema validation and unique SQL links. The authenticated integration suite uses disposable QA databases so that the demonstration seed remains unchanged.

| Verification area | Result |
|---|---|
| Local automated tests | 41 tests passed |
| Live MySQL checks | 11 quality-check groups passed |
| MongoDB validation | Type validation and duplicate-link protection passed |
| Application integration | Authenticated SQL/Mongo CRUD, RBAC, workflow, reports and statistics passed on disposable QA databases |
| Seed verification | Original statuses, fixed Mongo document IDs and CSE/ECE/ME scores restored for demonstration |

# 11 Sample Application Code and Demonstration

```
'use client';
import Statistics from './statistics.js';
import {matchesRegister} from '../lib/statistics.js';
import {useEffect,useState,useCallback} from 'react';
import Reports from './reports.js';
import {canEdit,transitions} from '../lib/policy.js';
const examples={
 research_paper:{journal_name:'Journal name',publication_year:2026,indexing:'Scopus',doi:''},
 patent:{patent_number:'IN202600001',patent_status:'published',country:'India'},
 workshop:{role:'attendee',duration_days:1},
 fdp:{program_name:'Faculty development programme',duration_days:5,mode:'hybrid'},
 book_chapter:{book_title:'Book title',publisher:'Publisher',isbn:''},
 mentorship:{program_name:'Mentor-Mentee Scheme',mentee_count:20},
 placement:{students_placed:80,total_eligible:100,top_recruiters:[]},
 lab_upgrade:{lab_name:'Computing lab',cost_inr:100000,equipment:[{item:'Workstation',quantity:1}]},
 budget:{allocated_inr:100000,spent_inr:90000,heads:[{head:'Equipment',amount_inr:90000}]},
 infrastructure:{facility:'Library',area_sqft:1000}
};
function template(kind,type,name){return {...(kind==='faculty'?{submission_type:type,authors:[{name:name||'Author name',affiliation:'PES University',is_internal:true}],k
const typeFor={'3.2.1':'patent','3.3.1':'research_paper','3.3.2':'book_chapter','2.4.1':'workshop','2.3.4':'mentorship','4.1.3':'lab_upgrade','5.2.1':'placement','6.4.1'
async function api(path,method='GET',body){
 const r=await fetch('/api/'+path,{method,headers:{'Content-Type':'application/json'},...(body?{body:JSON.stringify(body)}:{})});
 const data=await r.json();if(!r.ok){const e=new Error(data.error||'Request failed');e.status=r.status;throw e;}return data;
}
function EvidenceFields({value,onChange,disabled}){
 let doc;try{doc=JSON.parse(value)}catch{return <p role="alert">Evidence details could not be read.</p>}
 const label=k=>k.replaceAll('_',' ');
 const set=(key,val)=>onChange(JSON.stringify({...doc,[key]:val},null,2));
 const detail=(key,val)=>set('details',{...doc.details,[key]:val});
 const types=doc.submission_type==='workshop'||doc.submission_type==='fdp'?['workshop','fdp']:null;
 const enums={indexing:['Scopus','WoS','UGC-CARE','Other'],patent_status:['filed','published','granted'],role:['attendee','resource_person','organizer'],mode:['online','
 function list(key,value){
 const rows=Array.isArray(value)?value:[];
 const mutate=(index,next)=>detail(key,rows.map((r,i)=>i===index?next:r));
 return <div className="detail-array" key={key}><h4>{label(key)}</h4>{rows.map((row,i)=><div className="detail-array-row" key={i}>{typeof row==='string'?<input aria-labe
 }
```

*Figure 8. Next.js source code capture from app/page.js.*



```
export class AppError extends Error { constructor(message,status=400){super(message);this.status=status;} }
export const assert=(condition,message,status=400)=>{if(!condition)throw new AppError(message,status)};
export function integer(value,label='ID'){const n=Number(value);assert(Number.isSafeInteger(n)&&n>0,label+' must be a positive integer');return n;}
export function text(value,label,max=500){assert(typeof value==='string'&&value.trim().length>0&&value.length<=max,label+' is required (max '+max+' characters)');return
export function url(value){const s=text(value,'Evidence URL',500);let u;try{u=new URL(s)}catch{throw new AppError('Enter a valid evidence URL')};assert(['http:','https:'
export function kindConfig(kind){assert(['faculty','department'].includes(kind),'Unknown record type',404);return kind==='faculty'?{table:'Faculty_Submissions',id:'submi
export function canRead(user,row,kind){return ['Auditor','Admin'].includes(user.role)||(user.role==='Faculty'&&kind==='faculty'&&Number(row.faculty_id)===user.user_id)||
export function canEdit(user,row,kind){return user.role===(kind==='faculty'?'Faculty':'HOD')&&Number(row[kind==='faculty'?'faculty_id':'hod_id'])===user.user_id&&['Draft
export function transitions(user,row,kind){
 if(row.cycle_status!=='Active')return [];
 const owner=Number(row[kind==='faculty'?'faculty_id':'hod_id'])===user.user_id&&user.role===(kind==='faculty'?'Faculty':'HOD');
 if(owner&&row.status==='Draft')return ['Submitted'];
 if(owner&&row.status==='Rejected')return ['Draft'];
 if(row.status==='Submitted'&&((kind==='faculty'&&user.role==='HOD'&&Number(row.dept_id)===user.dept_id)||(kind==='department'&&user.role==='Auditor')))return ['Verified
 if(row.status==='Verified'&&user.role==='Auditor')return ['Approved','Rejected'];
 return [];
}
const finite=(o,k,min,max=Number.MAX_SAFE_INTEGER)=>{assert(typeof o[k]==='number'&&Number.isFinite(o[k])&&o[k]>=min&&o[k]<=max,k+' must be between '+min+' and '+max)};
const string=(o,k)=>{o[k]=text(o[k],k,2000)};
export function evidence(input,kind){
 assert(input&&typeof input==='object'&&!Array.isArray(input),'Evidence must be an object');
 const d=input.details;assert(d&&typeof d==='object'&&!Array.isArray(d),'details must be an object');
 // Whitelist type-specific fields; immutable IDs/owners are always supplied by the server.
 const type=kind==='faculty'?input.submission_type:input.metric_category;
 const specs={
 research_paper:{required:['journal_name','publication_year','indexing'],fields:['journal_name','publication_year','indexing','impact_factor','doi']},
 patent:{required:['patent_number','patent_status'],fields:['patent_number','patent_status','filing_date','country']},
 workshop:{required:['role','duration_days'],fields:['role','duration_days','attendee_count','certificate_url']},
 fdp:{required:['program_name','duration_days','mode'],fields:['program_name','organizer','duration_days','mode']},
 book_chapter:{required:['book_title','publisher'],fields:['book_title','publisher','isbn']},
 mentorship:{required:['program_name','mentee_count'],fields:['program_name','mentee_count','period']},
 placement:{required:['students_placed','total_eligible'],fields:['students_placed','total_eligible','highest_package_lpa','average_package_lpa','top_recruiters']},
 budget:{required:['allocated_inr','spent_inr'],fields:['allocated_inr','spent_inr','heads']},
 lab_upgrade:{required:['lab_name','cost_inr'],fields:['lab_name','cost_inr','vendor','equipment']},
 infrastructure:{required:['facility','area_sqft'],fields:['facility','area_sqft','capacity']}
 };
```

*Figure 9. Server-side RBAC and workflow source capture from lib/policy.js.*

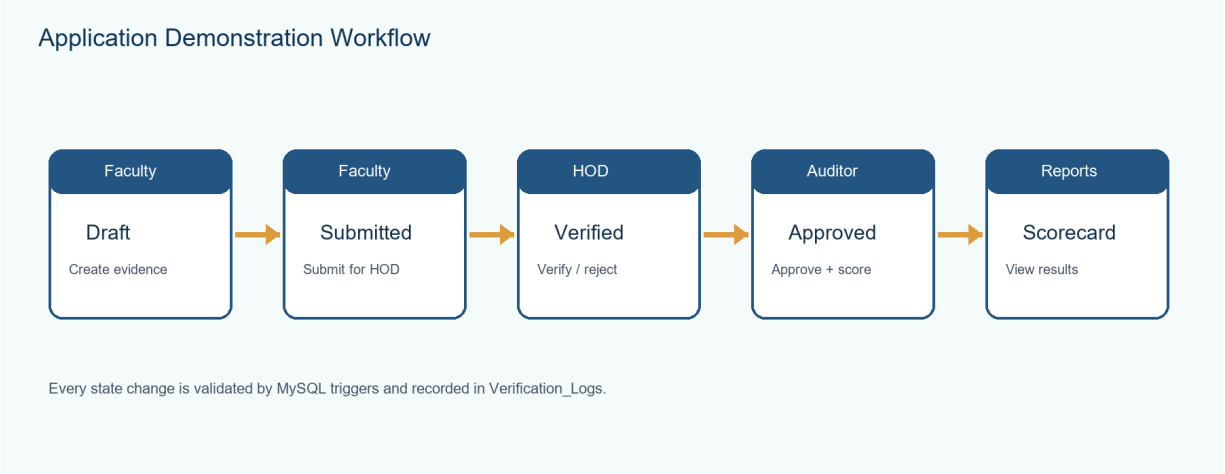
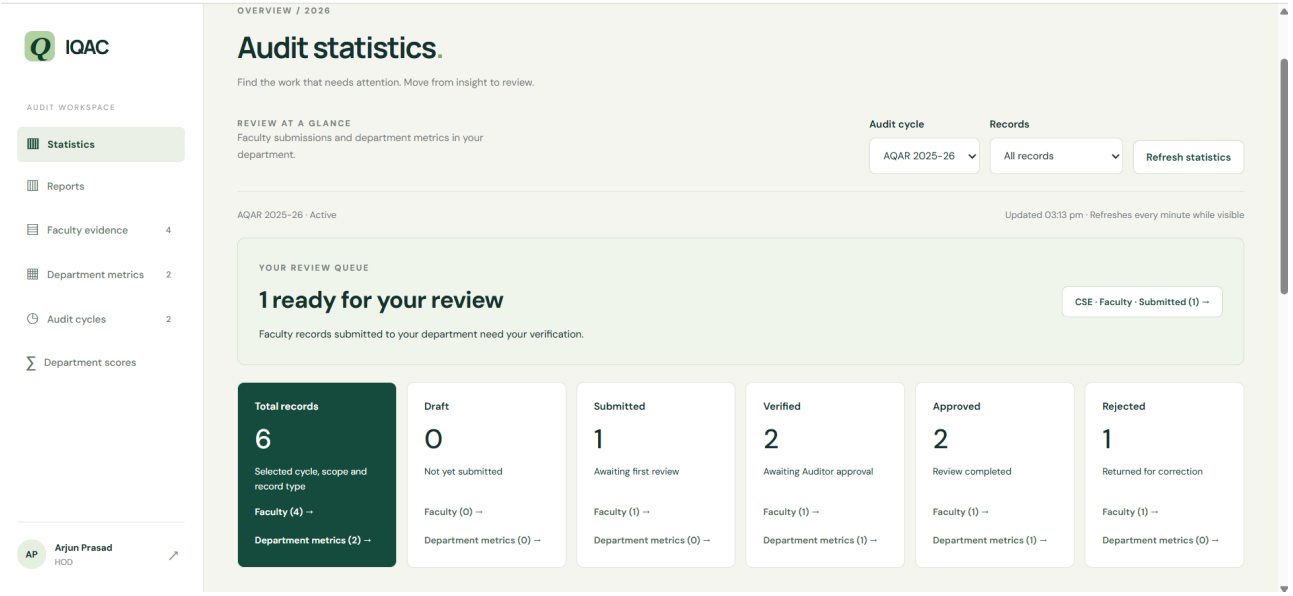
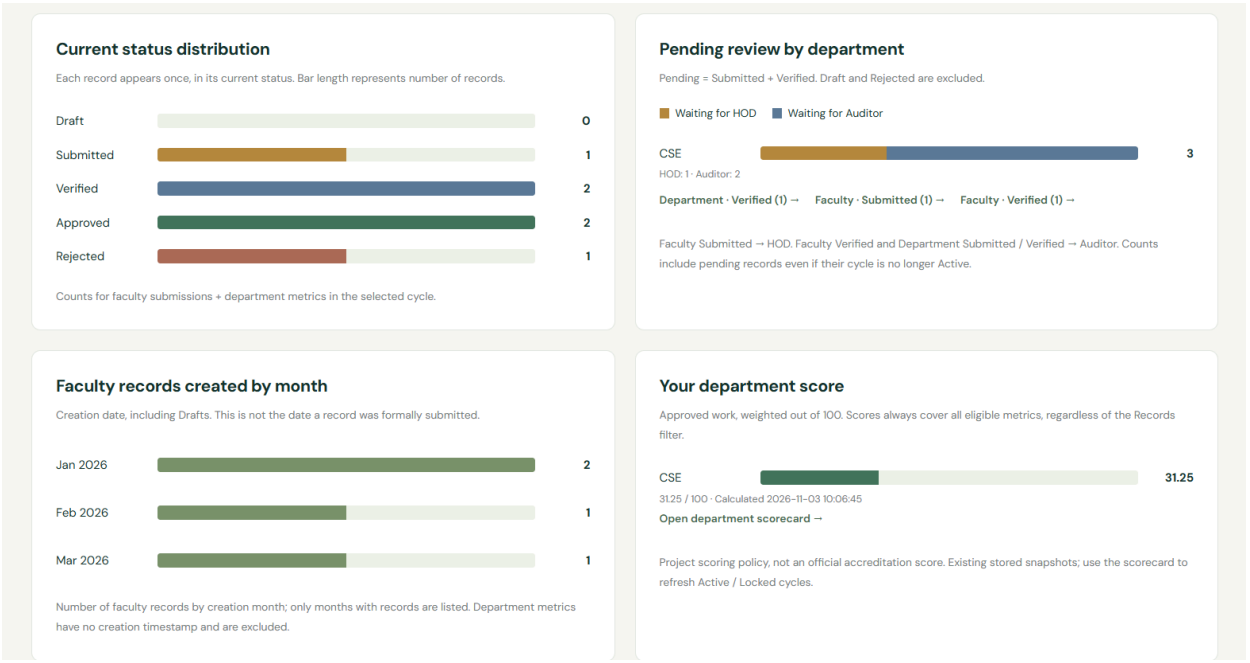



*Figure 10. Demonstration flow used in the live role-based walkthrough.*





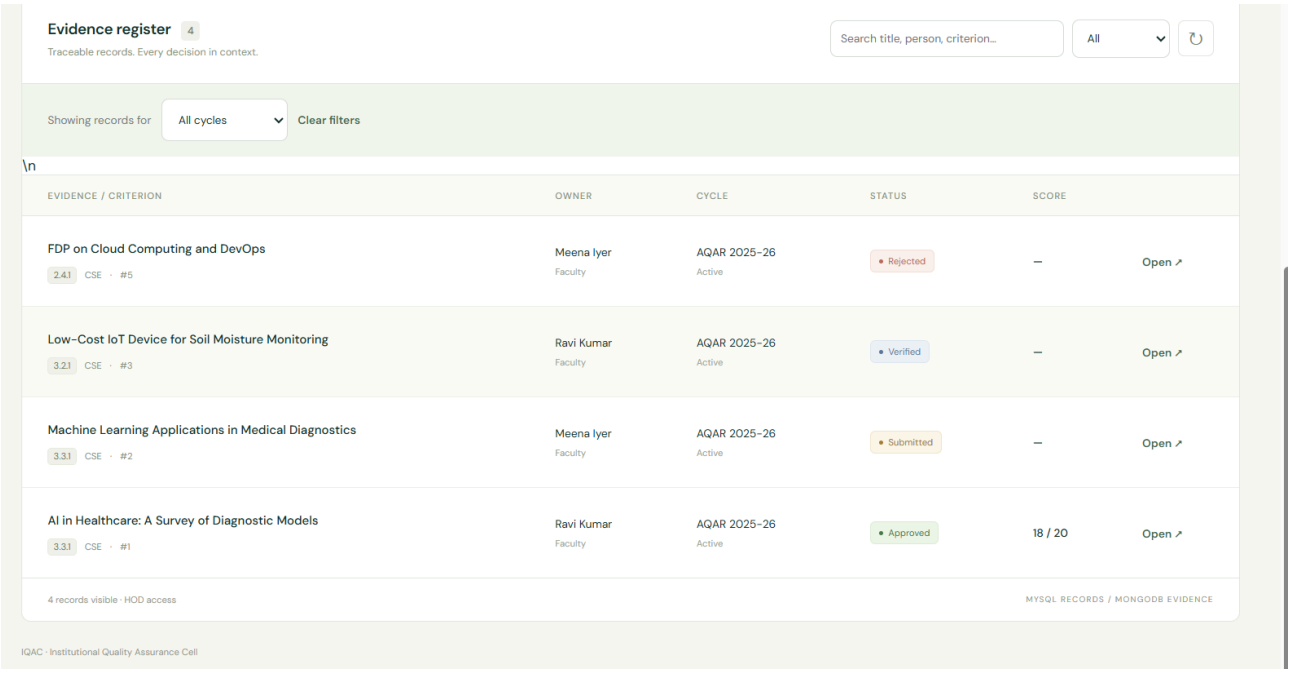




*Figure 11. Dashboard demonstration*

# 12 Evaluation Rubric Completion

| Criterion | Completion evidence | Status |
|---|---|---|
| Database design | Eight core tables, Chen ER diagram, relation mapping, keys and constraints | Complete |
| Database creation | MySQL DDL, MongoDB validators/indexes, technology justification | Complete |
| SQL operations | DML scripts, joins, nested/correlated/recursive queries and reports | Complete |
| Advanced concepts | Triggers, functions, procedure, CTE and MongoDB synchronization guard | Complete |
| Web integration | Next.js interface, API routes, MySQL and MongoDB integration | Complete |
| Authentication and authorization | Session login and four role-specific access policies | Complete |
| Documentation and testing | README, setup guide, lifecycle, report queries and automated checks | Complete |

# 13 Conclusion and Future Scope

The project demonstrates how a hybrid relational and document-database design can make an IQAC evidence workflow traceable, reviewable and easier to report. MySQL protects the workflow and calculations, while MongoDB accommodates varied evidence structures. The portal presents the same data through role-specific workspaces and reports.

A suitable future enhancement is a co-author credit model. At present, a faculty submission has one owning faculty member, even though the MongoDB evidence can list multiple authors. A future Faculty_Contribution relation could allocate approved credit across internal co-authors using an explicit policy, preventing first-submitter ownership from becoming an unfair scoring assumption.